PatchGuru: Patch Oracle Inference from Natural Language Artifacts

Thanh Le-Cong[1], Bach Le[2], Toby Murray[2], Cristian Cadar[3], Michael Pradel[4]
[1]Singapore University of Technology and Design (SUTD), Singapore
[2]University of Melbourne, Melbourne, Australia
[3]Imperial College London, London, United Kingdom
[4]CISPA Helmholtz Center for Information Security, Saarbrücken, Germany

***Abstract*—As software systems evolve, code changes may inadvertently introduce unintended behavioral changes. Validating patches against their intended behaviors remains challenging due to the lack of adequate and machine-checkable specifications: regression tests are often incomplete, while natural language (NL) descriptions of patch intent are informal and non-executable. This paper presents PatchGuru, the first automated technique that infers executable patch specifications from real-world pull requests (PRs). Given a PR, PatchGuru leverages large language models to distill developer intent from its NL artifacts and synthesizes an under-approximate yet practical form of patch specifications called *patch oracles*. The benefits of patch oracles are threefold: (1) they concentrate on behaviors affected by the patch, making their inference more tractable and their validation more efficient; (2) they are expressed as runtime assertions, enabling automated validation; and (3) they are written within comparison programs combining both pre- and post-patch versions, thus allowing cross-version properties to be specified. By comparing the behavior of pre- and post-patch versions of modified functions against these inferred oracles, PatchGuru iteratively refines the oracles, identifies inconsistencies when violations arise, filters them via a self-review mechanism, and eventually reports likely bugs to developers. We evaluate PatchGuru on 400 recent PRs from four widely used open-source Python projects. The evaluation shows that PatchGuru reports 39 warnings with a precision of 0.62, yielding 24 confirmed true positives, including 12 previously unknown bugs, 11 of which have already been fixed following our reports. Compared to the state-of-the-art patch validation technique Testora, PatchGuru detects 17 more bugs (24 vs. 7) while increasing precision from 0.32 to 0.62. PatchGuru also complements state-of-the-art LLM-based code review tools, detecting 12 bugs that Codex misses, while achieving substantially higher precision. PatchGuru has a reasonable average cost of around 8.9 minutes and $0.07 per PR. We envision PatchGuru as a complement to existing code review and regression testing practices, providing both explicit documentation and automated validation of patch intent.**

I. Introduction

Software systems continuously evolve to meet the changing needs of users and businesses. This evolution is primarily realized through patches that aim to fix bugs, introduce new functionality, improve performance, or refactor code. Ironically, patches, which are intended to improve the software, can inadvertently introduce bugs and even vulnerabilities, leading to catastrophic consequences. For example, a minor update in CrowdStrike caused a severe regression that disrupted approximately 8.5 million devices [1], and a small patch in OpenSSL caused havoc by introducing the HeartBleed security vulnerability [2]. Therefore, validating patches against their intended behavior before integration is crucial.

Patch validation faces a long-standing challenge: the lack of a precise, machine-checkable specification of intended patch behavior. Regression tests are a common partial substitute, but they are costly to maintain and often incomplete in practice [3]–[5]. Because they are largely not designed for individual patches, they may miss whether intended behavioral changes are actually realized. This "specification gap" lets buggy patches pass existing tests while introducing unintended behavior or preserving behavior meant to change.

A common proxy for intent is natural language (NL) artifacts, such as commit messages and pull request (PR) descriptions. Yet, these artifacts are informal and not directly machine-checkable for automated patch validation. As a result, developers rely on manual code review [6], [7], which is time-consuming, error-prone, and dependent on reviewer expertise. Moreover, most review comments focus on maintainability rather than functional correctness [6], allowing intent-misaligned buggy patches to slip into production and potentially cause regressions and vulnerabilities [8]. More recently, LLM-based code review tools have been adopted in practice [9]. However, similar to human reviewers, these tools primarily reason about code patches implicitly rather than explicitly validating their runtime behavior, and therefore cannot reliably determine whether a patch's actual behavior matches its stated intent [10]. Across these approaches, a fundamental limitation remains: they do not produce an explicit, machine-checkable specification of the behavioral change that a patch is intended to introduce. Such a specification is crucial for automatically validating intended patch behavior against actual runtime behavior, rather than inferring it indirectly from informal NL descriptions or reviewer judgments.

Recently, Testora [11] directly targets this specification gap, generating differential tests to expose behavioral differences between the pre- and post-patch versions and using an LLM to determine whether the differences are intended. However, as Testora relies on a differential oracle, it cannot identify bugs where a patch fails to implement an intended change and both versions exhibit the same behavior. More importantly, developer intent is represented implicitly through LLM judgments rather than as an explicit specification that can be validated, refined, or shared with developers. Thus, although Testora goes

beyond static code review by observing runtime behaviors, it still does not provide a machine-checkable specification of what the patch is supposed to change.

The need for the machine-checkable specification in patch validation has been articulated in a recent vision paper by Cadar et al. [12], which proposed *patch specifications* to formally specify the intended behavioral delta between pre- and post-patch program versions. However, patch specifications are designed to hold universally over all inputs. This makes them highly desirable, but difficult to construct in practice, as they require precise reasoning about the intended behavior of a code change across relevant inputs and edge cases. Consequently, patch specifications are typically written manually [12], [13], and no existing technique automatically infers them from real-world patches, limiting their practical adoption.

This paper presents PatchGuru, the first practical, automated technique for generating machine-checkable patch specifications from real-world patches, and checking them via automatically generated *comparison programs* [14]. PatchGuru infers what we call *patch oracles*, an under-approximate yet practical form of patch specifications that capture key behavioral properties of patches. We design patch oracles as *runtime assertions* paired with *concrete inputs* within a comparison program, which is a synthetic program that contains the pre- and post-patch versions of a modified function as independent functions, allowing for direct comparison of their behavior. The benefits of this design are threefold. First, patch oracles focus on behavior affected by the patch, making their inference more tractable and their validation more efficient compared to whole-program validation, e.g., regression testing or formal verification. Second, their representation as assertions enables direct execution within comparison programs, providing automated patch validation through dynamic analysis. Third, the use of comparison programs allows PatchGuru to specify cross-version properties between pre- and post-patch behaviors, which are essential for characterizing patch intent, yet difficult to express using traditional specification approaches.

Example. Consider a buggy patch that aims to change a function `f` from the identity function to one that returns its argument plus 1. PatchGuru automatically generates the following *comparison program*, embedding both versions as independent functions, together with a *patch oracle* consisting of runtime assertions paired with test inputs to be checked.

```
def pre_f(x): return x          # pre-patch version
def post_f(x): return x + 2     # post-patch version
test_inputs = [1, 2, 3]
for x in test_inputs:
    assert post_f(x) == pre_f(x) + 1
```

Executed over multiple inputs x, a violation of the oracle immediately exposes the inconsistency, i.e., the fact that the patched function mistakenly returns `x + 2` instead of `x + 1`. §II and Fig. 2 present a detailed real-world example.

To infer patch oracles, we utilize NL artifacts associated with patches, e.g., commit messages, descriptions, and discussions, which typically convey the developers' intent behind a change. Given a PR, PatchGuru distills relevant NL artifacts and leverages LLMs to synthesize a machine-checkable patch oracle and an associated comparison program. The inferred patch oracle is executed and iteratively refined by generalizing assertions and exploring edge cases. When assertion failures arise, PatchGuru adopts an LLM-as-a-judge to review the failures, confirm inconsistencies between the code changes and inferred oracles, and generate a report for developers.

Our evaluation applies PatchGuru to 400 real-world PRs from four popular and complex open-source projects, showing that it finds bugs that have remained undetected during code review and regression testing. In total, PatchGuru identifies 24 bugs, i.e., true inconsistencies between code changes and their intended behavior, split between 17 code bugs and 7 documentation bugs. Among the code bugs, 12 were previously unknown, all have been confirmed by developers, and 11 are already fixed following our reports. Compared to Testora [11], PatchGuru detects 17 more bugs (24 vs. 7) while reducing the false positive rate from 0.68 to 0.38. Compared to the code review feature of Codex CLI [15], a commercial LLM-based coding agent from OpenAI, PatchGuru achieves substantially higher precision and detects 12 bugs that Codex misses, showing that PatchGuru can complement existing LLM-based code review tools to improve bug detection and provide actionable warnings to developers. Additionally, in a fault injection campaign, PatchGuru-inferred oracles achieve a higher mutation score than existing regression tests (0.70 vs. 0.58), while combining the two achieves a mutation score of 0.81, demonstrating that PatchGuru-inferred oracles effectively complement existing regression tests. Finally, we show that the costs of using PatchGuru are acceptable for real-world deployment, with an average inference time of 8.9 minutes and an average LLM cost of USD 0.07 per PR.

In summary, this paper makes the following contributions:

- *Idea.* We introduce the idea of patch oracle inference from NL artifacts associated with patches, providing a novel way of bridging the specification gap in patch validation.
- *Technique.* We present PatchGuru, an automated technique that synthesizes and iteratively refines patch oracles as comparison programs.
- *Evaluation.* We conduct a large-scale evaluation of PatchGuru on 400 real-world PRs from popular open-source Python projects, demonstrating its effectiveness in inferring adequate patch oracles and identifying real bugs.
- *Artifacts.* We open-source our code and data [16].

II. Approach

A. Problem Statement

Before presenting our approach, we formally define the problem that PatchGuru aims to solve.

Input: PatchGuru takes as input a PR consisting of two components: (1) code changes $\mathcal{C}$, and (2) NL artifacts $\mathcal{D}$, including the PR description, commit messages, and related discussions that explain the rationale for the change. We currently focus on PRs that modify exactly one function, which simplifies the construction of comparison programs. We acknowledge this as a limitation and further discuss it in §IV.

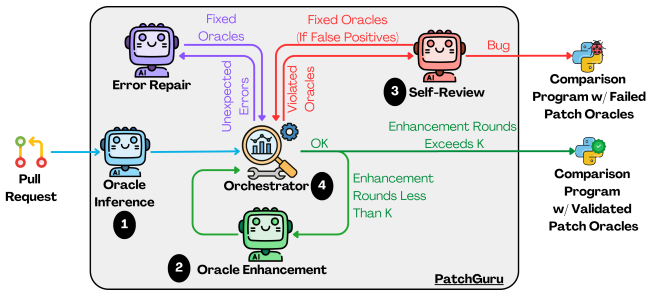


Fig. 1: Overview of PatchGuru's workflow.

Output: PatchGuru outputs a triplet $(\mathcal{O}, \mathcal{P}, \mathcal{R})$, where $\mathcal{O}$ is an inferred patch oracle comprising runtime assertions and test inputs that specify the intended relationship between pre- and post-patch behavior; $\mathcal{P}$ is a comparison program that validates both versions against $\mathcal{O}$; and $\mathcal{R}$ is a natural language report explaining detected inconsistencies. When an inconsistency is found, PatchGuru raises a warning and returns the triplet as reproducible evidence. We assume the pre-patch version is correct, as the goal is to detect patch-introduced bugs.

B. Overview

Fig. 1 presents an overview of PatchGuru's workflow, which consists of three main phases coordinated by an orchestrator. Given a PR, PatchGuru first employs an LLM-based inference module (①, §II-C) to derive an initial patch oracle from the PR's NL artifacts. The inferred oracle includes runtime assertions that encode the intended behavior described in the PR, together with test inputs that trigger these behaviors. If the behavior of the code patch is consistent with the inferred oracle, PatchGuru enters an iterative oracle enhancement phase (②, §II-D) to improve the completeness of the inferred oracle. During this phase, PatchGuru generalizes initial assertions to cover broader input domains, and synthesizes additional test inputs to explore edge cases. At any point, if inconsistencies are detected, PatchGuru triggers a self-review phase (③, §II-E), which applies an LLM-as-a-judge mechanism to distinguish true inconsistencies from false positives. Throughout the workflow, the orchestrator (④, §II-F) executes inferred oracles, monitors runtime behavior, repairs incorrect oracles, and orchestrates transitions between phases.

C. Oracle Inference

The goal of the oracle inference phase is to formalize the developer's intent, as expressed in the NL artifacts of a PR, into a machine-checkable patch oracle. The resulting oracle consists of runtime assertions paired with test inputs that characterize the intended behavior of the modified function. Specifically, we define three general assertion categories to capture the behavioral relationship between the pre-patch and post-patch versions of the function as follows: (1) *preserved behavior* assertions that specify identical behavior between the pre- and post-patch versions for certain inputs; (2) *changed behavior* assertions that specify modified behaviors for specific inputs; and (3) *new behavior* assertions that describe entirely new functionality added by the patch. Given a PR, we prompt an LLM to synthesize concrete assertions and corresponding test inputs through a three-step process: (1) collecting relevant NL artifacts, (2) extracting the code context, and (3) inferring patch oracles from the collected information.

1) Gathering Natural Language Artifacts: PatchGuru gathers NL artifacts associated with the PR, including the PR title, description, and developers' discussions (see Fig. 2(a) for an example) as they often contain explicit descriptions of the intended behavior and rationale behind the code changes. It also retrieves issues linked in the PR, as they may contain

(a) PR Context

PR Title: Fix: add file URL handling to the URL validator

PR Description: Requires a modicum of special handling due to hostnames being optional. Fixes #2249. [Issue #2249] Marshmallow's `URL` validator does not accept file URLs without host. For example, `file:///var/storage/somefile.zip` raises a `ValidationError`.

(b) Inferred Patch Oracle & Comparison Program

```
from marshmallow.fields import URL
from regex import search
from marshmallow.exceptions import ValidationError
def pre_fn(validator, url):
    # pre-patch implementation of fields.URL.__call__
def post_fn(validator, url):
    # post-patch implementation of fields.URL.__call__
def call(validator, pre_fn, post_fn, url):
    try:
        pre_res, pre_exc = pre_fn(validator, url), None
    except ValidationError as e:
        pre_res, pre_exc = None, e
    try:
        post_res, post_exc = post_fn(validator, url), None
    except ValidationError as e:
        post_res, post_exc = None, e
    return pre_res, pre_exc, post_res, post_exc
validator = URL(schemes=["https", "file"])
## PRESERVED: VALID URLS
valid_url = "http://example.com/path"
pre_res, pre_exc, post_res, post_exc = call(validator, pre_fn, post_fn,
    valid_url)
assert pre_exc is None and post_exc is None and pre_res == valid_url and
    post_res == valid_url, ("[PRESERVED] Both pre_fn and post_fn should
    accept valid URLs.")
## PRESERVED: INVALID URLS
invalid_url = "ftp://example.com/path"
pre_res, pre_exc, post_res, post_exc = call(validator, pre_fn, post_fn,
    invalid_url)
assert isinstance(pre_exc, ValidationError) and isinstance(post_exc,
    ValidationError), ("[PRESERVED] Both pre_fn and post_fn should
    reject invalid URLs.")
## CHANGED: FILE URLS
lower_file = "file:///etc/passwd"
pre_res, pre_exc, post_res, post_exc = call(validator, pre_fn, post_fn,
    lower_file)
assert isinstance(pre_exc, ValidationError) and post_exc is None and
    post_res == lower_file, ("[CHANGED] post_fn should accept file URLs
    while pre_fn rejects them.")
## CHANGED: FILE URLS (ENHANCED)
upper_file = "FILE:///etc/passwd"
pre_res, pre_exc, post_res, post_exc = call(validator, pre_fn, post_fn,
    upper_file)
assert isinstance(pre_exc, ValidationError) and post_exc is None and
    post_res == upper_file, ("[CHANGED] post_fn should accept file URLs
    (case-sensitive) while pre_fn rejects them.")
```

Error Message: `AssertionError: [CHANGED] post_fn should accept file URLs while pre_fn rejects them.`

Self-Review: The failing assertion is caused by the post-patch code rejecting uppercase `FILE:///...` URLs even though the PR intent is to support file URLs without hostnames. Conclusion: [BUG]

Fig. 2: Shortened real-world example based on a Marshmallow patch for function `fields.URL`. It shows (a) PR context and (b) comparison program (blue) and patch oracle (green: initial inference; orange: after enhancement) inferred by PatchGuru. The PR introduces a bug detected by PatchGuru.

TABLE I: PatchGuru's prompt design including tasks, key guidelines and inputs. See full prompts in [17].

Module	Task	Key guidelines	Inputs					
			NL[a]	ctx[b]	chg[c]	ora[d]	pro[e]	rpt[f]
Inference	Infer initial patch oracle and comparison program	Understand PR context and developer intent; derive invariant behaviors and intended changes; produce precise, executable assertions and comparison program with test inputs	✓	✓				
Enhancement	Generalize and broaden coverage of patch oracle	Generalize existing assertions; explore edge cases and boundary conditions; diversify inputs while keeping existing assertions intact	✓	✓		✓	✓	
Self-review	Classify assertion errors as bugs or false positives	Cross-check PR details, code changes, and failure evidence; distinguish true bugs from flawed tests; provide a clear conclusion and correction if needed	✓	✓	✓	✓	✓	✓
Repair	Fix runtime errors	Inspect the full traceback; check common errors; repair observed issues	✓	✓		✓	✓	✓

[a]NL artifacts (§II-C1); [b]code context (§II-C2); [c]chg = code changes; [d]ora = current patch oracle; [e]pro = comparison program; [f]execution report (§II-F1).

intent not explicitly stated in the PR description. For instance, in Fig. 2(a), the original PR description does not specify that `fields.URL` should accept file URLs without hostnames; this intent was instead documented in the referenced issue #2249. To do so, PatchGuru extracts issue URLs from the PR description via regular expression matching and retrieves the corresponding issue content via GitHub's REST API. As these references may contain irrelevant information, PatchGuru applies an LLM-based distillation to retain only details that directly relate to the intended changes, discarding unrelated information and tangential discussions.

2) Extracting Context of Modified Functions: PatchGuru also extracts relevant code context for the changed function from the codebase to provide additional information for oracle inference and ensure the executability of the inferred oracle. In particular, PatchGuru first identifies the modified function by analyzing the code changes in the PR, and then identifies dependencies (i.e., callees) of the modified function, which are necessary for executing the inferred patch oracle. For dependencies within the same file, PatchGuru parses the modified file into an abstract syntax tree and extracts top-level function and class definitions. For dependencies in external modules, PatchGuru analyzes import statements and retrieves the corresponding modules. If the modified function is a class method, PatchGuru also extracts the complete class definition to provide additional context and ensure the correct initialization of instance variables during oracle execution. For example, in Fig. 2(b), PatchGuru extracts the complete implementation of the `fields.URL.__call__` method, which is the modified function, along with its dependencies such as the `search` function from the `regex` library, the `ValidationError` class from the `marshmallow.exceptions` module, and the class definition of `URL` to provide necessary context for initializing the `URL` validator instance in line 18.

3) Patch Oracle Inference: Using the collected NL artifacts and code context, PatchGuru invokes an LLM with a structured prompt template, consisting of a task description, guidelines, and contextual inputs (summary in Table I; full template in [17]), to synthesize an initial patch oracle. A key design choice in our prompt is to provide step-by-step guidelines, inspired by the least-to-most prompting technique [18], decomposing the patch-oracle inference task into manageable subtasks to improve the quality and reliability of the generated oracles. In addition to the patch oracle, we also ask the LLM to jointly generate a comparison program to enable automatic validation of the generated oracle. Specifically, PatchGuru instructs it to generate the comparison program with placeholders for the pre- and post-PR implementations, which are later replaced with the actual implementations extracted from the codebase as described in §II-C2 to ensure executability. For example, Fig. 2(b) shows the comparison program and patch oracle inferred by PatchGuru for the PR in Fig. 2(a). The oracle asserts three behavioral properties: (1) valid URLs are accepted unchanged by both versions and returned as-is; (2) invalid URLs raise a `ValidationError` in both versions; and (3) file URLs without hostnames are rejected pre-patch but accepted post-patch.

D. Oracle Enhancement

Given the initial patch oracle, the oracle enhancement phase iteratively refines it to improve its bug detection capabilities. For instance, in Fig. 2, PatchGuru initially infers assertions that account only for lowercase file URLs, i.e., those beginning with "file://" (line 30). During the oracle enhancement phase, PatchGuru generalizes these assertions to include uppercase file URLs, e.g., those beginning with "FILE://" (line 34), improving oracle adequacy and exposing a previously unknown PR-introduced bug: the post-patch code fails to handle uppercase file URLs correctly.

To enhance the inferred oracle, PatchGuru invokes an LLM with a prompt that has a similar structure to the oracle inference prompt, but with a different task and guidelines (summary in Table I; full template in [17]). The new task and guidelines are designed to address the primary failure modes of initial oracles as identified in our preliminary analysis: they tend to be overly specific to the inferred example inputs (mitigated by generalizing existing assertions and diversifying test inputs) and may fail to capture boundary behaviors (mitigated by exploring edge cases and boundary conditions).

E. Self-Review

Once assertions are violated during execution, PatchGuru enters the self-review phase to determine whether these violations correspond to true inconsistencies (i.e., bugs) or false positives arising from incomplete or incorrect oracles. Specifically, for each failing assertion, PatchGuru invokes an LLM-as-a-judge with a self-review prompt (summary in Table I; full template in [17]) that instructs it to analyze the

Algorithm 1 Core algorithm of PatchGuru.

Input: PR $(\mathcal{C}, \mathcal{D})$ with code patch $\mathcal{C}$ and NL artifacts $\mathcal{D}$, maximum number of LLM calls M, maximum number of enhancement iterations N
Output: Patch oracle $\mathcal{O}$, comparison program $\mathcal{P}$, and validation report $\mathcal{R}$

1: $q \leftarrow 0$ ▷ Track LLM calls
2: $\mathcal{O}, q \leftarrow INFER(\mathcal{D})$ ▷ Infer initial patch oracle (§II-C)
3: $status, msg \leftarrow EXECUTE(\mathcal{O}, \mathcal{C})$ ▷ Execute patch oracle (§II-F1)
4: $iter \leftarrow 0$
5: **while** $iter < N$ **do**
6: **if** $status$ is `NO_VIOLATION` **then**
7: $\mathcal{O}, q_u \leftarrow ENHANCE(\mathcal{O}, \mathcal{D})$ ▷ Enhance oracle (§II-D)
8: $iter \leftarrow iter + 1$
9: **else if** $status$ is `ASSERTION_VIOLATION` **then**
10: $\mathcal{O}, \mathcal{R}, res, q_u \leftarrow REVIEW(\mathcal{O}, msg, \mathcal{D}, \mathcal{C})$ ▷ Self-review (§II-E)
11: **if** res is `TRUE_POSITIVE` **then**
12: **return** $\mathcal{O}, \mathcal{R}$
13: **end if**
14: **else if** $status$ is `UNEXPECTED_ERROR` **then**
15: $\mathcal{O}, q_u \leftarrow REPAIR(\mathcal{O}, msg, \mathcal{D})$ ▷ Repair errors (§II-F3)
16: **end if**
17: $q \leftarrow q + q_u$ ▷ update LLM call count
18: **if** $q \geq M$ **then**
19: **return** $\mathcal{O}, \varnothing$ ▷ Exceeded LLM call limit
20: **end if**
21: $status, msg \leftarrow EXECUTE(\mathcal{O}, \mathcal{C})$ ▷ Execute patch oracle (§II-F1)
22: **end while**
23: **return** $\mathcal{O}, \varnothing$ ▷ No inconsistencies found

NL artifacts, code context, code changes, and execution report to provide a verdict of either true positive or false positive, along with a detailed explanation supporting the verdict. In case of a false positive, PatchGuru also prompts the LLM to propose concrete enhancements to the oracle. In case of a true positive, PatchGuru raises a warning to notify developers of the detected inconsistency and generates a validation report that summarizes the findings. For instance, in Fig. 2, the LLM identifies that the assertion failure arises from the post-patch code's case-sensitive handling of file URLs, which contradicts the developer's intent. Thus, the LLM identifies the failure as a true inconsistency, i.e., a bug, and provides a detailed explanation.

F. Core Algorithm

The orchestrator manages the PatchGuru workflow by managing interactions among LLM modules and executing the inferred oracles. Given a set of oracles inferred or refined by the LLM, the orchestrator (i) constructs and executes a comparison program with the oracles and (ii) selects subsequent actions based on the execution outcome, including oracle enhancement, self-review, or error repair, following Alg. 1.

1) Constructing and Executing Comparison Programs: As mentioned in §II-C3, the patch oracle contains placeholders for the pre- and post-patch implementations. The orchestrator replaces these placeholders by extracting the relevant code functions from the pre- and post-patch versions of the modified file, based on the ranges of deleted and added lines. To avoid naming conflicts, the orchestrator renames the pre- and post-patch functions by appending the prefixes `pre_` and `post_`. Additionally, the orchestrator also integrates all necessary dependencies extracted in §II-C2 into the comparison program to ensure executability. By following this procedure, the orchestrator converts the inferred patch oracle into an executable comparison program to validate pre- and post-patch behavior. While our extraction may miss some dependencies due to the complexity of real-world codebases, PatchGuru handles them in the subsequent error repair phase (§II-F3). As isolated execution of arbitrary modified functions is challenging, PatchGuru may generate mock data types or stubs to make comparison programs executable, similar to ChangeGuard [14]. Such synthetic inputs can be unrealistic and may cause false positives, as discussed in §III-B.

Once an executable comparison program is constructed, the orchestrator proceeds to execute it while monitoring runtime behavior (Alg. 1). To this end, PatchGuru builds a Docker container that encapsulates the necessary runtime environment, including language interpreters, libraries, and dependencies required for executing the comparison program. This containerized approach ensures consistency and isolation during execution, mitigating potential conflicts with the host environment. During execution, the orchestrator captures detailed execution logs, including standard output, error messages, and stack traces. It also monitors the evaluation of assertions defined in the patch oracle, recording any violations or exceptions that occur. After execution, the orchestrator compiles a comprehensive report summarizing the execution results, including the status of each assertion (passed or failed), any runtime errors encountered, and relevant execution logs. This report is then relayed back to the respective LLM modules for oracle enhancement, self-review, or error repair.

2) Selecting Next Actions: After executing the comparison program, the orchestrator evaluates the results to determine the next action, considering several scenarios:

- *No Errors or Violations:* If all assertions pass without runtime errors, the orchestrator considers the pre- and post-patch behaviors consistent under the current oracle. PatchGuru then invokes the oracle enhancement phase (§II-D, Alg. 1, line 7) to further refine the oracle. If this phase reaches its iteration limit without detecting inconsistencies, PatchGuru terminates and reports the comparison program with the validated oracle.
- *Assertion Violations:* If any assertions fail during execution, the orchestrator triggers the self-review phase (§II-E; Alg. 1, line 10). The self-review process determines whether the violations represent true inconsistencies or false positives that require oracle enhancement. Note that, as PatchGuru also generates assertions on the pre-patch function to capture its intended behavior, assertion violations can also be raised on the pre-patch function. However, as we assume that the pre-patch function is correct (§II-A), any assertion violations on the pre-patch function are treated as *reasoning errors* made by the LLM and handled in the error repair phase (§II-F3, Alg. 1, line 15). This design enables PatchGuru to avoid misinterpreting the behavior of the pre-patch function, thereby providing more accurate patch oracles.

- *Unexpected Errors:* If any other errors occur during execution, the orchestrator categorizes these as execution errors and invokes the error repair module (§II-F3, Alg. 1, line 15) to address them. Specifically, we consider two main categories of execution errors: (1) *Syntax errors*, such as incorrect indentation, missing colons, or unmatched parentheses; (2) *Runtime errors* during comparison-program execution, such as `NameError` from undefined variables or functions, and `TypeError` from incompatible data-type operations.

3) Error Repair: When execution errors are detected by the orchestrator, PatchGuru invokes an LLM-based error repair module to diagnose and resolve these issues. As discussed above, PatchGuru focuses on three main categories of errors: (i) Reasoning Errors, which lead to assertion violations on the pre-patch function, (ii) Syntax Errors during compilation, and (iii) Runtime Errors during execution. To address these errors, PatchGuru prompts an LLM with a structured error repair prompt that guides the LLM to analyze the error message and the context of the generated code to identify the root cause and propose concrete fixes.

III. Evaluation

We answer the following research questions (RQs):

- **RQ$_1$**: Can patch oracles generated by PatchGuru detect real-world bugs?
- **RQ$_2$**: How adequate are the patch oracles generated by PatchGuru?
- **RQ$_3$**: What is the per-PR cost of PatchGuru?
- **RQ$_4$**: How do individual components of PatchGuru contribute to its performance?

A. Implementation Details and Experimental Setup

While PatchGuru is conceptually language-agnostic, we evaluate it by implementing a prototype for the Python ecosystem. We select Python due to its popularity and the availability of mature, large-scale open-source repositories. Moreover, LLMs have demonstrated strong capabilities in understanding and generating Python code [19]. We employ GPT-5-mini [20] as the underlying LLM as our preliminary experiments showed that it offers a reasonable trade-off between effectiveness and computational cost. As OpenAI does not allow configuring the temperature for GPT-5-mini, we use the default temperature setting hard-coded by OpenAI. We set per-call limits of 400,000 input tokens and 16,384 output tokens; in our evaluation, most runs are well below these bounds.

All experiments are conducted on an Ubuntu 22.04 LTS server with a 32-core AMD EPYC 9474F processor and 128 GB of RAM. To maintain computational efficiency, each test execution, including compilation and execution within Docker (as described in §II-F1), is limited to 1 h. Each phase of PatchGuru is individually capped at 5 LLM invocations per PR, with an overall budget of 20 LLM invocations per PR across all phases. This limit was empirically determined by our preliminary experiments to balance effectiveness and cost.

Dataset. To evaluate PatchGuru, we select four widely-used open-source Python projects: Keras (deep learning), Marshmallow (serialization), Pandas (data analysis), and SciPy (scientific computing). This selection aligns with the benchmark dataset used by our primary baseline Testora [11] while covering diverse application domains. From each project, we collect merged PRs and apply three filters. First, we exclude PRs that modify only documentation (files in documentation directories, code comments, README files). Second, we exclude PRs whose changes are confined exclusively to test files. Third, we retain only PRs that modify exactly one function. From the qualifying PRs, we select the 100 most recently merged PRs per project, yielding 400 PRs in total. The complete list of PR identifiers is available in the folder (`cache/pr_ids`) of our replication package [16].

Baselines. To evaluate the effectiveness of our approach, we compare PatchGuru against three baselines: Testora, LLM-based code review, and developer-written regression tests.

a) Testora (RQ$_1$): Testora [11] is a state-of-the-art patch validation technique that also checks Python code patches against NL artifacts in PRs. It generates tests and executes them independently on pre- and post-patch program versions to obtain their outputs for comparison. When differences arise between the outputs, Testora uses an LLM to determine whether observed behavioral differences are intended w.r.t. the NL artifacts; if not, it raises a warning. For a fair comparison, we sought the help of Testora's author to run it on our dataset of PRs using their recommended settings and GPT-5-mini as the underlying LLM (i.e., the same model used by PatchGuru). We compare against Testora in RQ$_1$ but not in RQ$_2$ because Testora generates test cases rather than explicit patch oracles, making it infeasible to compare their adequacy.

b) LLM-based code review (RQ$_1$): We also compare PatchGuru against OpenAI's Codex CLI [15], an industrial-strength, LLM-based code review tool. We select it as a baseline because it is (i) widely used, (ii) open-source, and (iii) developed by OpenAI, the same organization behind GPT-5-mini, our underlying LLM. Similar to Testora, we also configure Codex CLI to use GPT-5-mini and apply its `/review` feature to analyze the subject PRs.

c) Developer-written regression tests (RQ$_1$ and RQ$_2$): Developers commonly rely on regression tests to ensure that software evolution does not break existing functionality. Thus, to assess the quality of the patch oracles generated by PatchGuru, we compare them against the developer-written regression tests associated with each project. Due to SciPy's recent migration from `dev.py` to `spin`, we are only able to execute regression tests for the 29 most recent PRs.[1] To ensure an efficient comparison, we run only those tests that execute the function modified in the analyzed patch.

B. RQ$_1$: Effectiveness at Finding Real-World Bugs

We apply PatchGuru to the 400 PRs and manually analyze raised warnings to identify true inconsistencies between code

[1]The migration caused misconfigured default test commands and test failures, requiring costly version-specific customization to address.

TABLE II: Effectiveness of PatchGuru and Testora in detecting real-world bugs. #PRs, #Ora., #Warn, #TP, and Prec. denote the number of analyzed PRs, generated oracles, issued warnings, true positives, and precision.

Project	#PRs	PatchGuru				Testora		
		#Ora.	#Warn	#TP	Prec.	#Warn	#TP	Prec.
Keras	100	80	9	5	0.56	0	0	N/A
Marshmallow	100	83	7	4	0.57	8	2	0.25
Pandas	100	83	11	8	0.73	10	3	0.30
SciPy	100	90	12	7	0.58	4	2	0.50
Overall	**400**	**336**	**39**	**24**	**0.62**	**22**	**7**	**0.32**

patches and developer intent expressed in NL artifacts. To ensure accuracy, warnings were reviewed by the first author and cross-validated by two other authors; any disagreements are resolved through discussion. A true positive is a warning that correctly identifies such an inconsistency; any other warning is a false positive. During manual inspection, we find that some true positives exhibit desirable behavior despite being inconsistent with stated developer intent (e.g., inadvertently fixing an unrelated bug). We therefore classify true positives into: (i) *documentation bugs*, where documentation inaccurately describes the patch's unintended but desirable behavior, and (ii) *code bugs*, where the patch introduces unintended and undesirable behavior. We report code bugs that still exist in the latest version, but exclude documentation bugs because all dataset PRs are already merged and developers are unlikely to revise the PR descriptions after merging.

Overall Results. Table II summarizes the effectiveness of PatchGuru and Testora in detecting real-world bugs across the four studied projects. Out of 400 PRs from four projects, PatchGuru successfully infers patch oracles for 336 PRs, yielding a success rate of 84%. The success rate is also quite consistent across different projects, ranging from 80% (Keras) to 90% (SciPy). Unsuccessful inferences are due to (1) unresolvable runtime errors (33/64 cases) and (2) unresolvable LLM query errors (31/64), mostly caused by token limits when generating comparison programs and lack of adherence to the expected format. This limitation is expected given current LLM capabilities and the difficulty of generating executable comparison programs for arbitrary real-world patches [14]. We therefore restrict the remaining analysis to the 336 successfully inferred oracles. Using these oracles, PatchGuru raises 39 warnings across the four projects, of which 24 are true positives and 15 are false positives, yielding a 0.62 precision.

True Positives. Table III summarizes the 24 true positives identified by PatchGuru. Among these, 17 are code bugs and 7 are documentation bugs. Notably, of the 17 code bugs, 12 were still present in the latest version of these projects when PatchGuru discovered them, while the remaining 5 had been fixed independently by developers (4) or belonged to functions that were subsequently deleted in later versions (1). We reported the 12 previously unknown bugs to the respective project maintainers; as of this writing, maintainers confirmed all these bugs and fixed 11 of them. The only one remaining unfixed bug is in the SciPy project, in which maintainers acknowledged the issue and proposed a fix, but the fix has not yet been implemented. It is worth noting that all PRs analyzed by PatchGuru had already been merged into the main codebase, i.e., these issues were not detected by standard patch validation processes, including both regression testing and manual code review. This demonstrates PatchGuru's potential to improve software quality assurance by detecting real-world bugs missed by existing processes.

TABLE III: Real-world bugs detected by PatchGuru. PR refers to the pull request under analysis.

ID	Repo	PR	Kind	Status
1	Keras	20626	Code	Confirmed and fixed
2	Keras	20765	Code	Confirmed and fixed
3	Keras	20879	Documentation	–
4	Keras	20928	Code	Confirmed and fixed
5	Keras	20974	Code	Confirmed and fixed
6	Marshmallow	1399	Documentation	–
7	Marshmallow	2698	Code	Fixed independently
8	Marshmallow	2699	Code	Not in latest version
9	Marshmallow	2800	Code	Confirmed and fixed
10	Pandas	60828	Code	Fixed independently
11	Pandas	61054	Code	Confirmed and fixed
12	Pandas	61162	Documentation	–
13	Pandas	61183	Documentation	–
14	Pandas	61646	Code	Confirmed and fixed
15	Pandas	61946	Code	Fixed independently
16	Pandas	61966	Code	Confirmed and fixed
17	Pandas	62085	Code	Confirmed and fixed
18	SciPy	22213	Code	Confirmed and fixed
19	SciPy	22475	Code	Fixed independently
20	SciPy	22532	Documentation	–
21	SciPy	22989	Documentation	–
22	SciPy	23280	Documentation	–
23	SciPy	23341	Code	Confirmed and fixed
24	SciPy	23520	Code	Confirmed

We find that PatchGuru effectively identifies three bug types. First, it detects *incomplete patches* that fail to fully implement the intended behavior. In Pandas PR #61966, PatchGuru inferred the oracle `pre_out == post_out[1:-1]` and generated inputs showing that the patch correctly handled `str` categories but not `string` categories, a bug later confirmed by Pandas maintainers. This case also highlights the benefit of patch oracles on cross-version properties. Second, PatchGuru detects *regressions*. In Keras PR #20765, a fix for partial batch sizes in `TimeDistributed` caused the post-patch version to silently accept masks with mismatched timesteps; PatchGuru inferred that both versions should raise an error and exposed the regression. Finally, PatchGuru detects *documentation bugs* where NL artifacts inaccurately describe patch behavior. In Pandas PR #61162, the description stated that the patch enables `Series[bool]` indexing without specifying that it applies only to integer-indexed series. PatchGuru generated oracles covering both integer- and non-integer indexes, revealing that the post-patch version raises exceptions for the latter, a discrepancy between documented and actual behavior.

False Positives. To understand the current limitations of PatchGuru, we manually analyze all 15 false positives identi-

fied during our evaluation. This analysis reveals two primary root causes. First, in 8 cases, PatchGuru generates unrealistic inputs in the synthesized patch oracles that do not reflect actual usage scenarios, thereby producing false warnings. This issue is caused by limitations in PatchGuru's current comparison program synthesis and input generation strategies. Specifically, PatchGuru occasionally synthesizes unrealistic mock data types to enable the execution of comparison programs, or generates inputs that violate preconditions along the calling stack from top-level APIs to the modified functions. Second, in three cases, PatchGuru generates incorrect assertions, which misinterpret the developer intent. Notably, two cases arise because PatchGuru disagrees with developer intent as expressed in NL artifacts despite correctly inferring it. For example, in PR #60867, PatchGuru infers that the intended behavior is to return `frozenset` objects with curly braces even for empty sets, but assumes that an empty `frozenset` should be represented as `frozenset()` as opposed to `frozenset({})`, resulting in false warnings. In two additional cases, PatchGuru fails to capture environment-specific behavior, leading to incorrect oracles. For the remaining two cases, insufficient context prevents us from confirming true positives, so we conservatively classify them as false positives.

Comparison with Testora. As shown in Table II, Testora issued 22 warnings, of which 7 were true positives, yielding a precision of 0.32. In contrast, PatchGuru detected substantially more true bugs (24 vs. 7), achieved higher precision (0.62 vs. 0.32), and found 22 bugs missed by Testora. Moreover, 22 of the 24 bugs detected by PatchGuru are unique to PatchGuru and were not identified by Testora. This performance improvement arises from two factors. First, PatchGuru infers explicit patch oracles that check both unintended behavioral changes and intended behavior described in NL artifacts, whereas Testora relies on differential testing between pre- and post-patch versions. For the example in Fig. 2, both versions exhibit the same incorrect behavior for uppercase file URL schemes, causing Testora to miss the bug, while PatchGuru's inferred patch oracles capture the intended changes and detect the issue. Second, PatchGuru refines oracles using execution feedback and self-review, improving their alignment with developer intent. By contrast, Testora relies on LLM-based interpretation of behavioral differences, which is more prone to hallucination. This advantage is particularly evident by the higher precision of PatchGuru over Testora (0.62 vs. 0.32). Despite these advantages, Testora still detects 5 bugs missed by PatchGuru. These bugs are missed because PatchGuru either fails to resolve runtime errors or generate incomplete patch oracles. These results suggest that while PatchGuru significantly improves bug detection compared to Testora, there are still opportunities for further enhancing its capabilities.

Comparison with Codex CLI. Finally, we compare PatchGuru against the `/review` feature of Codex CLI [15]; the results are summarized in Table IV. Since it produces many warnings per PR, manually labeling all warnings across all 400 PRs is infeasible. We therefore evaluate it in two parts: (i) we run it on the 24 PRs where PatchGuru detected a bug to see how well PatchGuru complements Codex; and (ii) we randomly sample 10 PRs per project from the remaining PRs and inspect all warnings to estimate Codex's precision.

TABLE IV: Effectiveness of Codex Code Review on the 24 buggy PRs detected by PatchGuru and on 40 randomly sampled PRs (10 per project). #Warn and #TP denote issued warnings and true positives, i.e., actual bugs, respectively.

Project	24 buggy PRs			40 sampled PRs		
	#Warn	#TP	Precision	#Warn	#TP	Precision
Keras	5	1	0.2	7	0	0.0
Marshmallow	4	4	1.0	2	0	0.0
Pandas	8	4	0.50	7	2	0.29
SciPy	6	3	0.50	7	0	0.0
Overall	23	12	0.52	23	2	0.09

Complementarity of PatchGuru to Codex Code Review. On the 24 PRs where PatchGuru detected a bug, Codex raised 23 warnings with 12 true positives (precision 0.52), missing 12 of the 24 bugs found by PatchGuru. This demonstrates that PatchGuru's patch-oracle-based approach catches bugs that Codex misses, showing the potential for PatchGuru to complement existing LLM-based code review tools.

Precision of Codex Code Review. On 40 randomly sampled PRs, Codex raised 23 warnings but only 2 true positives, yielding a precision of 0.09 and requiring substantial manual triage. We attribute this advantage to PatchGuru's use of patch oracles, which check whether a patch satisfies its intended behavior, whereas Codex Code Review relies on LLM-based reasoning that remains prone to false positives due to limited deep code-semantic understanding [10], [21], [22]. Overall, these results indicate that PatchGuru can complement existing LLM-based code review tools by improving bug detection and producing more actionable warnings.

C. RQ_2: Adequacy of Patch Oracles

Besides detecting real-world bugs, another crucial aspect of patch oracles is their adequacy, which directly impacts their effectiveness in validating code patches. In RQ_2, we evaluate the adequacy of patch oracles generated by PatchGuru using mutation testing [23], [24], a well-established approach for assessing test quality. Specifically, we leverage Mutmut [25], a mutation testing framework for Python, to generate mutants with all default mutation operators. Mutants are generated on the *post-patch* version to simulate patch-induced bugs. Test adequacy is measured by the mutation score, i.e., proportion of mutants killed, with higher scores indicating better adequacy.

Results. Fig. 3 compares the distribution of mutation scores achieved by PatchGuru-generated oracles, developer-written regression tests and their combination across the four studied projects. Overall, generated patch oracles are more adequate than developer-written regression tests, achieving 21% higher overall mutation scores (0.70 vs. 0.58). A Wilcoxon signed-rank test confirms that the improvements are statistically significant for Keras, Pandas, and SciPy (p-value < 0.05). Moreover, combining PatchGuru-generated oracles with developer-

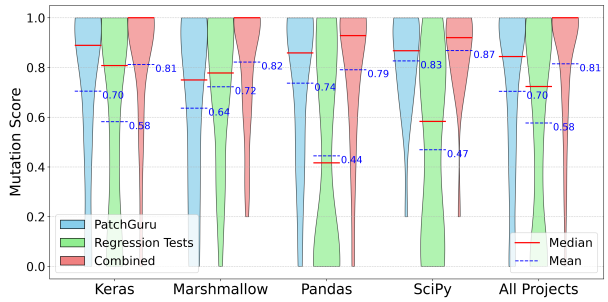


Fig. 3: Distribution of mutation scores for PatchGuru-generated oracles and developer-written regression tests.

written regression tests substantially improves the mutation score, to an overall value of 0.81. Median scores of the combined oracles even reach 1.0 in Pandas and Marshmallow. These results show that PatchGuru can effectively complement existing regression tests.

D. RQ_3: Efficiency

In RQ_3, we evaluate the computational cost of PatchGuru using two metrics: the analysis time per PR and the token usage of LLM calls. Because LLM costs vary over time and across providers, we measure input and output token counts, which directly correlate with incurred costs.

Execution Time. On average per checked PR, our approach takes 8.9 minutes to complete the entire analysis. Fig. 4a illustrates the distribution of execution times across all analyzed PRs. We can see that most PRs are analyzed within 20 minutes, with a few outliers taking longer due to the complexity of the patches or the need for multiple iterations of oracle enhancement. We can also see that the distribution of execution times is relatively consistent across different projects with average times ranging from 6.9 minutes (Marshmallow) to 10.4 minutes (Pandas). These results suggest that PatchGuru can analyze real-world PRs efficiently and within a reasonable time frame, making it feasible for practical CI/CD deployment.

Monetary Costs. On average per checked PR, PatchGuru utilizes 35,666 input tokens and 30,230 output tokens, equivalent to approximately USD 0.009 and USD 0.06, respectively, in LLM calls. Figures 4b and 4c illustrate the distribution of token usage across all analyzed PRs. We can see that most PRs require fewer than 60K input tokens and 50K output tokens. As for execution times, the token usage distribution is relatively consistent across different projects, with average input tokens ranging from 27,644 (Marshmallow) to 42,115 (SciPy) and average output tokens ranging from 24,846 (Marshmallow) to 32,694 (Keras). Based on the pricing of OpenAI's GPT-5-mini model as of June 2026, the total cost per PR is estimated to be approximately USD 0.07. We believe this cost is acceptable for practical use, especially considering the benefits of early bug detection and improved patch validation.

E. RQ_4: Ablation Study

RQ_4 conducts an ablation study to assess the contribution of individual components of PatchGuru to its overall performance. Specifically, we evaluate two modules: (i) the oracle enhancement module (§II-D) and (ii) the self-review module (§II-E). We create two ablation variants of PatchGuru by removing each component in turn and evaluating them on the same set of 400 PRs. The results are reported in Table V.

TABLE V: Ablation study results. * indicates estimated values.

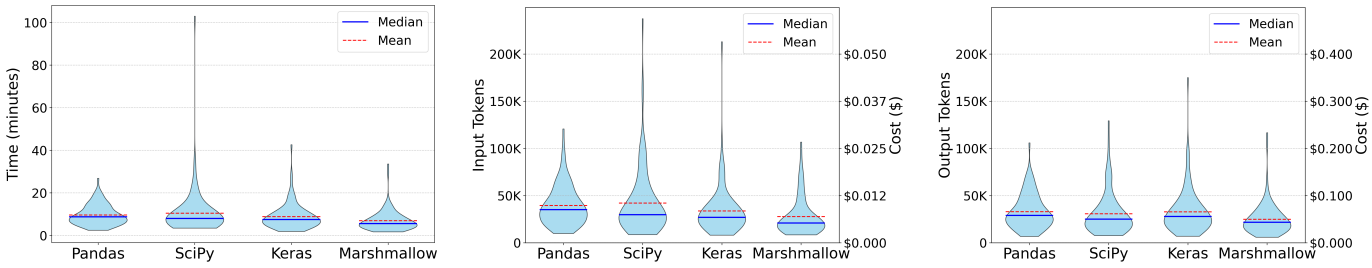
Method	#Warnings	#Bugs	Precision	Mutation Score
PatchGuru	39	24	0.62	0.70
w/o oracle enhancement	18	12	0.67	0.64
w/o self-review	195	25*	0.13*	N/A

PatchGuru without Oracle Enhancement. We first disable the oracle enhancement module and directly use the initial patch oracles generated by the LLM without further refinement. As shown in Table V, removing this module significantly reduces PatchGuru's effectiveness in detecting real-world bugs: the number of warnings drops from 39 to 18, and true positives decrease from 24 to 12. Therefore, the oracle adequacy also decreases, with the mutation score dropping from 0.70 to 0.64. Although precision slightly increases from 0.62 to 0.67, the bug detection is considerably affected. These results demonstrate that the enhancement module is essential for improving oracle quality by generalizing patch oracles and generating comprehensive test inputs.

PatchGuru without Oracle Self-Review. Then, we disable the self-review module and directly report all detected inconsistencies, yielding 195 warnings across 400 PRs, making exhaustive manual inspection impractical. We therefore estimate results by partitioning warnings based on the full PatchGuru's self-review outcomes: for warnings classified as "bug", we reuse the inspection results from RQ_1; for warnings filtered out, we randomly sample five per project, manually inspect them, and extrapolate to the full set. Based on this estimation, we found that disabling the self-review module causes a substantial increase in false positives, leading to a sharp drop in precision from 0.62 to 0.13. Of the 20 sampled warnings filtered by the self-review module, 19 were confirmed as false positives upon manual inspection with common false-positive patterns including incorrect comparison operators, invalid assumptions about pre-patch behavior, incorrect invariants, and improper input initialization. Although the estimated number of detected bugs increases slightly from 24 to 25, this gain comes at the cost of an overwhelming number of false positives, rendering the tool impractical for developers.

IV. Threats to Validity

Internal Validity. First, our manual classification into true and false positives is subject to human error; we mitigate this threat through cross-validation among authors and independent confirmation from project maintainers. Second, PatchGuru assumes the pre-patch function is correct; while this may cause false negatives when the pre-patch itself is buggy, it is consistent with PatchGuru's goal of detecting bugs *introduced* by the patch. Finally, LLM non-determinism threatens reproducibility [26]. However, we argue that this is inherent in



(a) Time (b) LLM input tokens (c) LLM output tokens

Fig. 4: Execution time and token usage per PR.

LLMs and reflects the variability that practitioners encounter in real-world usage. We minimize this risk by conducting a large-scale evaluation on 400 PRs across four projects and providing all intermediate artifacts in the replication package [16].

External Validity. Our benchmarks are not representative of all software projects. We mitigate this by selecting popular Python projects from diverse domains following Testora [11]. However, PatchGuru currently targets Python projects, which may limit the generalizability of our findings to other languages. Although its core approach is language-agnostic, supporting additional languages may require substantial engineering effort. Finally, PatchGuru currently handles only single-function patches, which simplifies comparison program construction. Extending it to multi-function patches is non-trivial because it requires accurately modeling interactions among modified functions and their dependencies. We acknowledge this as a limitation and leave it as future work.

Construct Validity. We use mutation score as a proxy for oracle adequacy. While mutation testing is a widely accepted technique for assessing test suite quality [23], [24], mutation score may not perfectly correlate with real-world fault detection capability. To mitigate this threat, we complement mutation score with evaluation on real-world PRs, which provides a more direct assessment of PatchGuru's effectiveness in practice. The effectiveness of PatchGuru also depends on LLM capabilities; due to limited funding, we evaluated only GPT-5-mini. Future work could assess performance across diverse LLMs and configurations.

V. Related Work

Reasoning about Code Changes. As software evolves, reasoning about code changes is crucial for maintaining software quality. The most relevant line of work to PatchGuru is on patch specification [12], [13] and software change contracts [27], which capture the intended behavior of code changes but require manually provided specifications. PatchGuru addresses this by automatically inferring patch oracles, an under-approximate yet practical form of patch specifications, from NL artifacts, enabling automated validation with minimal human intervention. PatchGuru is also closely related to recent work on reasoning about code changes using learning-guided execution and LLMs [11], [14]. Compared to ChangeGuard [14], which introduced the idea of comparison programs, we go beyond determining whether a code change introduces any behavioral differences but instead reason about the consistency of the intent expressed in the NL artifacts and the code change. Compared to Testora [11], our formulation not only checks whether PRs that introduce unintended behavioral changes, but also validates that the intended changes are correctly implemented as demonstrated in §III. PatchGuru is also related to just-in-time bug prediction [28]–[32] which estimates defect likelihood from historical data. However, recent works [29], [30], [33] show that these approaches have limited practical effectiveness due to class imbalance and noisy SZZ labels. More broadly, automated testing techniques have been used to target patches [5], [34]–[37]. While these techniques effectively generate tests that exercise code changes, they do not address the oracle problem, which is essential for determining correctness. In principle, they are complementary to PatchGuru and could be combined with it to generate tests that are then validated against inferred patch oracles.

Test Oracle Problem. Software testing relies on test oracles to determine the correctness of program outputs, yet such oracles are often incomplete or absent [38]. Prior work addresses this challenge by inferring oracles from API documentation, including metamorphic relations [39] and exceptional behavior [40]; or using deep learning models and LLMs [41], [42]. Other approaches infer program specifications that serve as oracles through dynamic invariant inference [43]–[45] or LLM-based techniques [21], [46]. Despite their effectiveness, these techniques primarily validate overall program behavior rather than behavior introduced or modified by code changes. This formulation is ill-suited to large, rapidly evolving systems because it requires comprehensive behavioral coverage, which is often infeasible in practice, and continual oracle updates, which impose substantial maintenance costs. In contrast, PatchGuru infers patch oracles that target only the behavior affected by a change, enabling more efficient validation.

VI. Conclusion

Software patches are essential for system evolution but remain a major source of bugs and vulnerabilities due to a lack of machine-checkable specifications capturing developer intent. In this paper, we introduced PatchGuru, an automated technique that infers machine-checkable patch oracles from natural language artifacts using LLMs and dynamic analysis.

Our evaluation on 400 real-world PRs shows that PatchGuru identifies 24 real inconsistencies, including 12 previously unknown code bugs. The approach imposes reasonable costs of 8.9 minutes and $0.07 per PR, suitable for real-world deployment. We envision PatchGuru as a complement to existing development practices by providing machine-checkable behavioral documentation and enabling early detection of unexpected behaviors missed by traditional approaches.

REFERENCES

[1] Wikipedia contributors, "2024 CrowdStrike-related IT outages," 2024. [Online]. Available: https://en.wikipedia.org/wiki/2024_CrowdStrike-related_IT_outages

[2] "Heartbleed bug," http://heartbleed.com/, Apr. 2014.

[3] P. D. Marinescu, P. Hosek, and C. Cadar, "Covrig: A framework for the analysis of code, test, and coverage evolution in real software," in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA'14)*, Jul. 2014, pp. 93–104.

[4] T. Bailey and C. Cadar, "Code, test and coverage evolution in mature software systems: Changes over the past decade," in *Proceedings of the IEEE International Conference on Software Testing, Verification, and Validation (ICST'25)*, Apr. 2025, pp. 210–220.

[5] Z. Zhou, M. Paltenghi, M. Kim, and M. Pradel, "Change and cover: Last-mile, pull request-based regression test augmentation," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE'26)*, 2026.

[6] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," *2013 35th International Conference on Software Engineering (ICSE)*, pp. 712–721, 2013.

[7] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, "An empirical study of the impact of modern code review practices on software quality," *Empirical Software Engineering*, vol. 21, no. 5, pp. 2146–2189, 2016.

[8] F. Khoshnoud, A. R. Nasab, Z. Toudeji, and A. Sami, "Which bugs are missed in code reviews: An empirical study on SmartSHARK dataset," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 137–141.

[9] U. Cihan, V. Haratian, A. İçöz, M. K. Gül, Ö. Devran, E. F. Bayendur, B. M. Uçar, and E. Tüzün, "Automated code review in practice," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2025, pp. 425–436.

[10] H. Y. Lin, C. Liu, H. Gao, P. Thongtanunam, and C. Treude, "CodeReviewQA: The code review comprehension assessment for large language models," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 9138–9166.

[11] M. Pradel, "Testora: Using natural language intent to detect behavioral regressions," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE'26)*, 2026.

[12] C. Cadar, D. Schemmel, and A. Sharma, "Patch specifications via product programs," in *2023 IEEE/ACM 11th International Conference on Formal Methods in Software Engineering (FormaliSE)*. IEEE, 2023, pp. 39–43.

[13] A. Sharma, D. Schemmel, and C. Cadar, "P^3: Reasoning about patches via product programs," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 2654–2680, 2025.

[14] L. Gröninger, B. Souza, and M. Pradel, "ChangeGuard: Validating code changes via pairwise learning-guided execution," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 936–956, 2025.

[15] OpenAI, "Codex CLI," https://developers.openai.com/codex/cli, OpenAI, 2025, accessed: 2026-06-30.

[16] Anonymous, "Replication package for "PatchGuru: Patch oracle inference from natural language artifacts"," https://github.com/thanhlecongg/PatchGuru, 2026.

[17] ——, "Online appendix for PatchGuru: Patch oracle inference from natural language artifacts," https://anonymous.4open.science/r/PatchGuru-13B9/appendix.pdf, 2026, accessed: 2026-06-30.

[18] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. V. Le, and E. H. Chi, "Least-to-most prompting enables complex reasoning in large language models," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WZH7099tgfM

[19] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *CoRR*, vol. abs/2107.03374, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[20] OpenAI, "Introducing GPT-5," https://openai.com/index/introducing-gpt-5/, OpenAI, 2025, accessed: 2026-06-30.

[21] T. Le-Cong, B. Le, and T. Murray, "Can LLMs reason about program semantics? a comprehensive evaluation of LLMs on formal specification inference," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 21 991–22 014. [Online]. Available: https://aclanthology.org/2025.acl-long.1068/

[22] C. Liu, Y. Chen, and R. Jabbarvand, "Assessing coherency and consistency of code execution reasoning by large language models," *In Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE'26)*, 2026.

[23] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE transactions on software engineering*, vol. 37, no. 5, pp. 649–678, 2010.

[24] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: an analysis and survey," in *Advances in computers*. Elsevier, 2019, vol. 112, pp. 275–378.

[25] A. Hovmöller, "Mutmut: a Python mutation testing system," https://kodare.net/2016/12/01/mutmut-a-python-mutation-testing-system.html, Dec. 2016, accessed: 2026-06-30.

[26] J. Sallou, T. Durieux, and A. Panichella, "Breaking the silence: the threats of using LLMs in software engineering," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, ser. ICSE-NIER'24. New York, NY, USA: Association for Computing Machinery, 2024, p. 102–106. [Online]. Available: https://doi.org/10.1145/3639476.3639764

[27] J. Yi, D. Qi, S. H. Tan, and A. Roychoudhury, "Software change contracts," *ACM Trans. Softw. Eng. Methodol.*, vol. 24, no. 3, May 2015. [Online]. Available: https://doi.org/10.1145/2729973

[28] T. Hoang, H. K. Dam, Y. Kamei, D. Lo, and N. Ubayashi, "DeepJIT: an end-to-end deep learning framework for just-in-time defect prediction," in *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 2019, pp. 34–45.

[29] Z. Zeng, Y. Zhang, H. Zhang, and L. Zhang, "Deep just-in-time defect prediction: how far are we?" in *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*, 2021, pp. 427–438.

[30] D. Nguyen, T. Le-Cong, T. H. M. Le, M. A. Babar, and Q.-T. Huynh, "Toward realistic evaluations of just-in-time vulnerability prediction," in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2025, pp. 1–10.

[31] D. Nguyen, M. Tran-Duc, T. Le-Cong, T. H. M. Le, M. A. Babar, and Q.-T. Huynh, "VulGuard: An unified tool for evaluating just-in-time vulnerability prediction models," in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2025, pp. 945–949.

[32] H. Perl, S. Dechand, M. Smith, D. Arp, F. Yamaguchi, K. Rieck, S. Fahl, and Y. Acar, "VCCFinder: Finding potential vulnerabilities in open-source projects to assist code audits," in *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, 2015, pp. 426–437.

[33] Y. Fan, X. Xia, D. A. Da Costa, D. Lo, A. E. Hassan, and S. Li, "The impact of mislabeled changes by SZZ on just-in-time defect prediction," *IEEE transactions on software engineering*, vol. 47, no. 8, pp. 1559–1586, 2019.

[34] T. Kuchta, H. Palikareva, and C. Cadar, "Shadow symbolic execution for

testing software patches," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 27, no. 3, pp. 1–32, 2018.

[35] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 2329–2344.

[36] T. E. Kim, J. Choi, K. Heo, and S. K. Cha, "DAFL: Directed grey-box fuzzing guided by data dependency," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4931–4948.

[37] K.-K. Ma, K. Yit Phang, J. S. Foster, and M. Hicks, "Directed symbolic execution," in *International Static Analysis Symposium*. Springer, 2011, pp. 95–111.

[38] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE transactions on software engineering*, vol. 41, no. 5, pp. 507–525, 2014.

[39] A. Blasi, A. Gorla, M. D. Ernst, M. Pezzè, and A. Carzaniga, "MeMo: Automatically identifying metamorphic relations in Javadoc comments for test automation," *Journal of Systems and Software*, vol. 181, p. 111041, 2021.

[40] A. Goffi, A. Gorla, M. D. Ernst, and M. Pezzè, "Automatic generation of oracles for exceptional behaviors," in *Proceedings of the 25th international symposium on software testing and analysis*, 2016, pp. 213–224.

[41] E. Dinella, G. Ryan, T. Mytkowicz, and S. K. Lahiri, "TOGA: A neural method for test oracle generation," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2130–2141.

[42] S. B. Hossain, R. Taylor, and M. Dwyer, "Doc2OracLL: Investigating the impact of documentation on LLM-based test oracle generation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 1870–1891, 2025.

[43] T. Nguyen, D. Kapur, W. Weimer, and S. Forrest, "Using dynamic analysis to discover polynomial and array invariants," in *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 2012, pp. 683–693.

[44] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The Daikon system for dynamic detection of likely invariants," *Science of computer programming*, vol. 69, no. 1-3, pp. 35–45, 2007.

[45] T. Le-Cong, D.-M. Luong, X. B. D. Le, D. Lo, N.-H. Tran, B. Quang-Huy, and Q.-T. Huynh, "Invalidator: Automated patch correctness assessment via semantic and syntactic reasoning," *IEEE Transactions on Software Engineering*, vol. 49, no. 6, pp. 3411–3429, 2023.

[46] M. Endres, S. Fakhoury, S. Chakraborty, and S. K. Lahiri, "Can large language models transform natural language intent into formal method postconditions?" *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1889–1912, 2024.